OSD 与系统联调

从零手搓 Step4 教程

2026 FPGA 创新设计赛道（安路）赛题二

视觉方向 • Step4_osd_integration

读者：零基础队员（建议先读完 Step1～3 教程）
目标：把检测结果画回画面——文字/准星/目标框，并完成整机联调
前置要求：会 Verilog、懂 Step2 检测链与 Step3 闭环的接口
预计学习时间：10～14 天（每天 2～3 小时）

本教程与你手上的代码是什么关系

Step2 在” 看”、Step3 在” 动”；Step4 把这一切**画给你看**——
左上角状态行、绿色准星、红色目标框。评委看到的画面从这里来。

目录

第一部分　从零学习教程 **3**

1　你在造什么 **3**
1.1　一句话 . . . 3
1.2　整机长什么样 . . . 3
1.3　做完是什么效果 . . . 3

2　阶段 0：点阵文字的原理（1～2 天） **3**
2.1　字就是一个位图 . . . 4
2.2　练习 . . . 4

3　阶段 1：坐标系统与叠加判决（2 天） **4**
3.1　自己数像素：x/y 计数器 . . . 4
3.2　叠加判决：一层” 该不该变色” 的组合逻辑 . . . 4
3.3　优先级即接线 . . . 5
3.4　练习 . . . 5

4　阶段 2：把数字写到屏幕上（2～3 天） **5**
4.1　二进制转 ASCII . . . 5
4.2　拼一条状态行 . . . 5
4.3　第二行遥测与跑马灯 . . . 5
4.4　练习 . . . 6

5　阶段 3：跨时钟域拿目标包（2 天） **6**
5.1　问题：检测结果” 随时” 到达 . . . 6
5.2　解法：影子寄存器 + 帧首换装 . . . 6
5.3　135bit 目标包位图（接口唯一权威定义在 IF-VISION.md） . . . 6
5.4　练习 . . . 7

6　阶段 4：整机集成（2～3 天） **7**
6.1　vision_top：把五个模块拼起来 . . . 7
6.2　” 包与帧拍异序” 的处理 . . . 7
6.3　练习 . . . 7

7　阶段 5：怎么证明画面是对的（1～2 天） **7**
7.1　逐像素金标准比对 . . . 7
7.2　练习 . . . 8

8　阶段 6：跑通测试（1～2 天） **8**
8.1　一键回归与怎么读结果 . . . 8

第二部分　工程实现详解 **9**

9　工程总览 **9**

9.1　文件清单 9

9.2　三时钟域与两条跨域河 9

10 模块详解：font_rom_8x16（生成文件） **9**

11 模块详解：osd_track v3 **10**

11.1 v1 到 v3 的演进 10

11.2 影子寄存器组：撕裂的结构性根治 10

11.3 文本构建：包到达拍的一次性算术 10

11.4 跑马灯的算术 10

11.5 准星与双框：带符号差 + 边缘四线 10

11.6 输出 mux 与颜色 11

12 模块详解：vision_top v2 **11**

12.1 算法切换：帧边界平稳换源 11

12.2 135bit 包拼装与保留位 11

12.3 控制路径：缓存 + 帧拍消费 11

12.4 声光反馈（给嵌入式方向的接口） 12

13 测试平台设计 **12**

13.1 tb_osd_track：整帧逐像素 12

13.2 tb_vision_top：五条整机断言 12

13.3 tb_closed_loop：相机模型驱动 TPG 12

14 验证结果与联调复盘 **12**

14.1 回归矩阵 12

14.2 ”永远 LOST” 事故复盘（本步骤最大缺陷） 13

14.3 Step4 真缺陷清单 13

15 上板挂载说明（给上板组） **13**

16 小结：Step4 的可复用产出 **13**

结语 **14**

总结与下一步 **15**

Step4 学到了什么 15

下一步学什么 15

从零学习教程

1 你在造什么

1.1 一句话

在视频画面上叠一层”仪表盘”：系统状态、目标十字、包围框，再把检测、云台、显示拼成一台完整的机器。

1.2 整机长什么样

```
                  +-----------------------------------------+
96bit 视频流 --->  |  直通出（原样给显示链路，零扰动）          |
       |           |                                          |
       +--> detect_top（检测）
                 |----> 135bit 目标包 --CDC--> osd_track --+--> OSD 叠加流
                 |----> 23bit 坐标    --CDC--> gimbal_ctrl --> pwm_servo
                 |----> frame_tick    --pulse_sync------^
```

三个角色，三个时钟域：检测/OSD 用像素时钟，云台/PWM 用 50MHz。跨域只走两座桥（pulse_sync、cdc_4phase，都是 Step1 的存货）。

1.3 做完是什么效果

屏幕左上角出现状态行：

```
TRK0 LOCK X0560Y0320          <- 跟踪组 0 / 已锁定 / 目标坐标
D02 B+00 C064 M0000           <- 延迟帧数 / 亮度 / 对比度 / 运动像素数
2026 FPGA CONTEST - ANLOGIC ...  <- 底部跑马灯
```

画面里：目标套红色框，中心绿色十字准星，运动目标还有橙色框

学完 Step4 你能做到

- 解释点阵字库的原理（8x16 ROM，一个字节一行）
- 用组合逻辑写出文字区/准星/框的叠加判决
- 把二进制坐标变成屏幕上的十进制字符
- 说清目标包跨域为什么要”影子寄存器 + 帧首换装”
- 用”逐像素金标准比对”证明整帧画面是对的

2 阶段 0：点阵文字的原理（1～2 天）

2.1 字就是一个位图

8x16 点阵字库：每个字符 8 像素宽、16 行高，每行一个字节。比如字母 A 的第 3 行可能是 `8'h3C`（二进制 00111100）——中间 4 个像素亮。整个字库存成 ROM，地址是 `{字符 ASCII 码，行号}`，读出 8 bit 就是这一行的 8 个像素。

```
addr = {char, row}          // 12bit: 8bit 字符码 + 4bit 行号
bits = font_rom[addr]       // 8bit, MSB = 最左像素
on   = bits[7 - (px % 8)]   // 当前像素在字节里的位置
```

为什么字库是”生成”的

手写 40 个字符 x 16 行的 case 表既痛苦又容易错。工程里字库由 Python 从 OpenCV 的 Hershey 字体渲染生成（`gen_vectors.py`），可复现、可重新生成——**生成文件禁止手改**，和 Step2 的倒数表同一条纪律。

2.2 练习

1. 在方格纸上手画 8x16 的”L” 点阵，写出 16 个字节的十六进制
2. 思考：MSB= 最左像素，如果搞反了会看到什么？
3. 算一下：1280x720 画面里，16 像素高的文字最多能放几行不重叠？

3 阶段 1：坐标系统与叠加判决（2 天）

3.1 自己数像素：x/y 计数器

OSD 模块收到的是 1 像素/拍的流，自己维护 `px/py` 计数器就知道当前画到哪了：

```
if (I_valid) begin
    if (I_sof)      px <= 1;      py <= 0;      // 帧首
    else if (I_eol) px <= 0;      py <= py + 1; // 行尾换行
    else            px <= px + 1;
end
```

valid 门控意味着：流里有消隐气泡（valid 拉低）时计数器自然停走，**不需要任何行场同步变换电路**——这是 vpix 流自带 sof/eol 的好处。

3.2 叠加判决：一层”该不该变色”的组合逻辑

每个像素问四个问题，按优先级决定颜色：

```
文字区:  py 在 [16,32) 且 px 在 [16,176)   -> 查字库，点亮点用白色
准星:    到 (cx,cy) 的横竖臂，臂宽 3..12   -> 绿色
目标框:  px 是 minx/minx+1/maxx-1/maxx 且 py 在框内 (或对称) -> 红色
运动框:  同上，用运动 bbox，mv=1 时生效     -> 橙色
都不是:  原样输出底图
```

准星为什么要”中空”

十字臂取 `|dx| 在 [3,12]` 而不是 [0,12]——中心 3 像素留空，否则准星正中把目标中心盖住，人眼反而看不清锁定点。这种细节在金标准渲染器里和 RTL 逐位对齐，两边一致才算对。

3.3 优先级即接线

实现上就是一个嵌套三元运算符（优先级从上到下）：

```
O_data <= on_txt  ? WHITE  :
          on_cross ? GREEN  :
          on_box   ? RED    :
          on_mbox  ? ORANGE : I_data;
```

3.4 练习

1. 写出”目标框只画 2 像素厚”的 x 边缘条件（提示：minx 和 minx+1）
2. 手推：目标在 (100,100)，哪些像素会被画成准星臂？
3. 思考：为什么判决全部用组合逻辑、只在输出寄存一拍？

4 阶段 2：把数字写到屏幕上（2～3 天）

4.1 二进制转 ASCII

坐标寄存器里是 11 bit 二进制数（比如 560），屏幕要显示字符’5”6”0’（ASCII 53、54、48）。转换就是反复除 10 取余：

```
function [31:0] to_digits4(input [13:0] val);
    d3 = 48 + val / 1000;   r = val % 1000;   // 千位
    d2 = 48 + r / 100;      r = r % 100;      // 百位
    d1 = 48 + r / 10;                          // 十位
    d0 = 48 + r % 10;                          // 个位
endfunction
```

48 是字符’0’ 的 ASCII 码。**为什么敢用除法：**这段逻辑每帧只在”包到达”时跑一次，不在像素流水线上——时序充裕，面积换省事完全正确。

4.2 拼一条状态行

20 字符缓冲在包到达时一次性写满：

```
位置：  0  1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17 18 19
内容：  T  R  K  0  (空) L  O  C  K  (空) X  0  5  6  0  Y  0  3  2  0
        ^^^^^ 算法名     ^^^^^ 锁定态     ^^^^ cx 四位十进制 ^^^^ cy
```

锁定但没目标时坐标四位替换成’—-’（0x2D），一眼可辨。

4.3 第二行遥测与跑马灯

- `D02`：延迟可视化——用”当前显示帧号 - 包里带的采集帧号”算出系统延迟帧数（进阶要求（3））

- `B+00 C064`：亮度/对比度当前值（扩展要求（4））
- `M0000`：运动检测命中像素数（进阶要求（4））
- 跑马灯：64 字符消息，每帧右移 2 像素，取模循环——纯计数器 + ROM

4.4 练习

1. 手推 to_digits4(560) 每一步的商和余
2. 如果画面 60fps，跑马灯每帧 2px，扫完整个消息一圈要几秒？
3. 思考：为什么坐标显示固定 4 位（补零）而不是变长？

5 阶段 3：跨时钟域拿目标包（2 天）

5.1 问题：检测结果”随时”到达

检测链在**帧末消隐期**吐出目标包（135bit），OSD 在**像素时钟**域逐拍画图。两边时钟相同（本工程共用像素时钟），但包到达时刻可能在 **一帧的中间**——直接更新绘制寄存器，会出什么事故？

撕裂：一帧里上半用旧坐标、下半用新坐标

目标在第 300 行时包到达，绘制寄存器立即换新值——第 0～299 行画的是旧位置的框，第 300～719 行画新位置的框。画面上出现**两个半截框**。这就是”撕裂”，Step4 v3 专治它。

5.2 解法：影子寄存器 + 帧首换装

双缓冲思想：包随时写**影子**寄存器组；绘制寄存器只在 `I_sof` 拍（新帧第一个像素）从影子**整组换装**：

```
if (pack_valid) begin          // 随时到，只写影子
    s_cx <= pack[...]; s_cy <= pack[...]; ...
end
if (I_valid && I_sof) begin    // 帧首一次换装
    cx <= s_cx; cy <= s_cy; ...
end
```

整帧画图期间绘制寄存器**恒定**——撕裂在结构上不可能发生。代价是结果最多滞后一帧显示，而 OSD 本来就是帧率更新，无感。

5.3 135bit 目标包位图（接口唯一权威定义在 IF-VISION.md）

```
[134]    valid          [65:58]  fid (采集帧号)
[133]    algo (0 跟踪 1 边缘) [57]    mot_valid
[132:130] preset 阈值组   [56:43]  mot_cnt
[129:119] cx             [42:32]  mot_minx
[118:108] cy             [31:21]  mot_maxx
[107:97]  minx           [20:11]  mot_miny
[96:86]   maxx           [10:1]   mot_maxy
[85:76]   miny           [0]      保留 (必须填 0)
[75:66]   maxy
```

bit0 保留位不是装饰

包拼接时少填一位，整包上移一位，OSD 读 valid 恒 0——画面永远显示 LOST。本项目真出过这个事故（详见 Part II 复盘）。规定：**发送端必须把保留位补齐，位偏移只在接口文档定义一处**。

5.4 练习

1. 手算：cx=560, cy=320，在包里各落在哪些 bit？
2. 若把 cx 从 [129:119] 错解成 [130:120]，读出的 cx 会偏大还是偏小？
3. 思考：为什么换装选 sof 而不是 vsync 信号？

6 阶段 4：整机集成（2～3 天）

6.1 vision_top：把五个模块拼起来

- **直通优先**：96bit 入口原样出（O_tdata=I_tdata），显示链路行为与官方例程完全一致；
- **检测分支**：detect_top 拿到坐标后拼 135bit 包；
- **云台分支**：23bit 坐标包 + 帧拍分别跨到 50MHz 域；
- **OSD 分支**：串化后的 1ppc 流 + 目标包；
- **声光**：led_buzzer（锁定绿灯/丢失红灯闪/锁定沿蜂鸣）。

6.2 ”包与帧拍异序”的处理

跨到 50MHz 域后，**坐标包和帧拍到达顺序不保证**（两个独立 CDC）。如果帧拍先到而包还没到，云台会用到旧坐标——可接受（1 帧滞后）；但**包先到、帧拍后到**时，包不能丢。工程用”缓存 + 帧拍消费”：

```
if (ctrl_valid_dst) begin      // 包到了：缓存，记下 valid
    ctrl_hold     <= ctrl_pack_dst;
    ctrl_vld_hold <= ctrl_pack_dst[22];
end
if (tick_pwm)                  // 帧拍到：消费一次，清 valid
    ctrl_vld_hold <= 1'b0;
```

gimbal_ctrl 每帧只看一次 `ctrl_vld_hold`——无论谁先到，消费时拿到的都是完整的一套。

6.3 练习

1. 画出 vision_top 里三个时钟域各住着哪些模块
2. 思考：直通输出如果插了一拍延迟，显示会怎样？（提示：和官方链路比对）
3. ctrl_hold 为什么不用双缓冲？

7 阶段 5：怎么证明画面是对的（1～2 天）

7.1 逐像素金标准比对

”看起来有框”不算数。Python 端有一个 `render_osd()`——和 RTL 绘制规则**逐位对齐**的纯软件渲染器（同样知道字库、框内缩、十字中空）。验证方法：

```
RTL 跑 5 帧 -> 整帧转储 (921600 像素/帧, 24bit)
Python 渲染同 5 帧 -> 逐像素比对 -> 全部一致才算 PASS
```

为什么这个方法是质量基线

肉眼只能看” 有没有框”，看不出” 框左移了 1 像素” 或” 第 37 行少画了一点”。逐像素比对连**一个像素的偏差**都逃不掉——Step4 联调抓出的全部 3 个真缺陷（包宽、旧位、字段溢出）都是它定位的，没有一处靠肉眼。

7.2 练习

1. 算一算：5 帧 x 921600 像素 x 24bit 的转储文件多大？
2. 思考：如果字库生成器改了一个字符的点阵，哪两边要同步重新生成？

8 阶段 6：跑通测试（1～2 天）

8.1 一键回归与怎么读结果

```
cd D:\fpga-ptz-tracker\tools
python run_all.py tb_osd_track tb_vision_top tb_closed_loop

--- tb_osd_track ...
    # FRAME0 OK hash= 一致    <- 整帧 921600 像素逐位一致
    ==> PASS

--- tb_vision_top ...
    # PASSTHRU OK           <- 直通逐拍一致
    # FRAME4 OK             <- OSD 帧与金标准一致
    # DUTY x=1428 y=1500    <- 云台朝目标方向动，Y 轴死区正确
    ==> PASS

--- tb_closed_loop ...
    TB_RESULT: PASS settle_frame=12 final_err=-8
    ==> PASS                <- 闭环 12 帧锁定
```

Step4 学习清单（全部打勾就过关）

- 能画 8x16 字库的取址与点亮过程
- 能写出文字/准星/框的判决条件并解释优先级
- 能解释影子寄存器为什么能消撕裂
- 能背出 135bit 包的字段顺序，说清保留位的教训
- 能解释” 逐像素金标准比对” 为什么比肉眼强
- 跑通 3 个测试并读懂输出

工程实现详解

这一部分写什么

仓库里真实做出来的 Step4 工程（RTL 共 1231 行：字库 684 + osd_track 313 + vision_top 234）逐模块讲透，外加三次联调迭代的完整复盘。对应目录：`03_vision/step4_osd_integration/`。

9 工程总览

9.1 文件清单

文件	类型	行数	职责
`font_rom_8x16.v`	生成	684	40 字符 ASCII 子集点阵（脚本生成）
`osd_track.v`	RTL	313	OSD 叠加层 v3（三行文本 + 双框 + 双缓冲）
`vision_top.v`	RTL	234	整机顶层 v2（检测 +OSD+ 云台 + 声光）
`tb/tb_osd_track.v`	测试	126	OSD 整帧逐像素比对
`tb/tb_vision_top.v`	测试	99	整机 5 断言
`tb/tb_closed_loop.v`	测试	95	闭环在环收敛
`sim_out/*.png`	证据	—	OSD 渲染对比图（金标准 vs RTL）

9.2 三时钟域与两条跨域河

```
pix 域 (74.25MHz):  检测链 / OSD / 显示路径
pwm 域 (50MHz):     gimbal_ctrl / pwm_servo
跨域信号:
   frame_tick    pulse_sync (单 bit)
   23bit 坐标包  cdc_4phase
   135bit 目标包 cdc_4phase (同域备用, 接口兼容异域)
```

一条贯穿全项目的设计规则

任何跨域信号只能走 pulse_sync 或 cdc_4phase 两座桥，禁止手搓两级同步器。本步的 135bit 包就是 `cdc_4phase #.WIDTH(135)` 的一次参数化复用——Step1 写好的桥，Step4 直接搬。

10 模块详解：font_rom_8x16（生成文件）

字库 ROM 覆盖 OSD 用到的 40 个 ASCII 字符（大写字母、数字、空格、`- + % . :` 等）。生成链路：

```
cv2.putText(Hershey 字体) -> 8x16 阈值化点阵 -> case 表 RTL
地址 {char[7:0], row[3:0]} -> 该行 8bit (MSB = 最左像素)
```

为什么选 8x16：16 像素高在 720p 上恰好是 45 行文字的量级，评委在演示视频里看得清；8 像素宽让字符地址 = 像素 x 右移 3 位，判决逻辑零乘法。

11 模块详解：osd_track v3

11.1 v1 到 v3 的演进

版本	内容
v1	单行文字 + 准星 + 单框，68bit 包（IF-VISION v1.0.0）
v2	配合 Step5 拓展：包扩到 135bit，三行文本 + 跑马灯 + 运动框
v3	消撕裂：影子寄存器组 + sof 拍整组换装（vsync 语义）

11.2 影子寄存器组：撕裂的结构性根治

v3 的核心是**两组一模一样的寄存器**：

- **影子组**（`s_*`）：`pack_valid` 拍写入，时刻可能在帧中间；
- **绘制组**（无前缀）：只在 `I_valid && I_sof` 拍从影子整组拷贝，帧内再也不会变。

绘制判决只读绘制组——无论包何时到，一帧之内的坐标/文字/框 **是同一份快照**。已知残留：sof 拍本身（像素 (0,0)）仍用旧值绘制一拍，但 OSD 起始在 x=16，无视觉影响（RTL 注释里明确记了这笔账）。

11.3 文本构建：包到达拍的一次性算术

```
txt1_s[0..2] = pack[133] ? "EDG" : "TRK"      // 算法名
txt1_s[3]    = 48 + preset                     // 阈值组号
txt1_s[5..8] = pack[134] ? "LOCK" : "LOST"    // 锁定态
txt1_s[11..14] = to_digits4(cx) 或 "----"      // 坐标四位
txt2: "D" lat " B" ±bb " C" cc " M" mmmm       // 遥测行
```

延迟显示 `lat = disp_cnt - fid`：包里带的是**采集时刻**的帧号，显示端用当前帧号一减，就是这套链路的真实延迟帧数（进阶（3）的可视化）。

11.4 跑马灯的算术

```
mar_off = disp_cnt * 2                  // 每帧 2px
mar_pos = px - TXT_X0 + mar_off         // 相对位置 + 滚动偏移
mar_ci  = mar_pos[8:3] % 63             // 字符索引 (63 字符取模)
on_mar  = 字形位 [7 - mar_pos[2:0]]      // 字节内位选
```

63 字符 x 8 像素 = 504 像素周期，2px/帧即 252 帧（约 4.2 秒）滚一圈。

11.5 准星与双框：带符号差 + 边缘四线

```
dx_c = $signed(px) - $signed(cx);   // 带符号差，一切判决的基础
ax = (|dx_c| <= 12) && (|dx_c| >= 3);   // 臂: 3..12 (中心 3px 留空)
```

```
on_cross = tv && ((ay && dx_c==0) || (ax && dy_c==0));
// 框：四条边各 2px 厚
x_edge = (px==minx) || (px==minx+1) || (px==maxx-1) || (px==maxx);
on_box = tv && ((x_edge && in_by) || (y_edge && in_bx));
```

带符号差是框类判决的唯一正确姿势

如果用无符号减法算 px - cx，目标左半边的像素差全部回绕成巨大正数，左半边框直接消失——和 Step3 闭环跑飞是同一类根因。本项目已两次被”无符号减法”咬，写进了方法论。

11.6 输出 mux 与颜色

优先级一个表达式写完（文字 > 准星 > 目标框 > 运动框 > 底图）：

```
O_data <= on_txt1|on_txt2 ? 24'hFFFFFF :   // 文字 白
          on_mar   ? 24'h00FFFF :          // 跑马灯 青
          on_cross ? 24'h00FF00 :          // 准星 绿
          on_box   ? 24'hFF0000 :          // 目标框 红
          on_mbox  ? 24'hFF8000 : I_data;  // 运动框 橙
```

12 模块详解：vision_top v2

12.1 算法切换：帧边界平稳换源

进阶（1）要求多算法切换。工程做法：**边缘检测常开**（并行分支），切换只动一个 mux 的选择端，且**在帧首拍锁存**：

```
if (I_tvalid && I_tuser)          // 只在帧首采样
    algo_q <= I_algo_edge;
// 之后一整帧 mux 固定选检测流或边缘流
```

帧中间切会怎样？mux 前两条流水线延迟不同，切换拍前后像素来自 **两条链的不同进度**，画面错位——帧边界锁存让切换永远发生在”两帧之间”，视觉无缝。

12.2 135bit 包拼装与保留位

```
wire [134:0] pack_vision = {t_valid, I_algo_edge, I_preset_idx,
    t_cx, t_cy, t_minx, t_maxx, t_miny, t_maxy, t_fid,
    mo_valid, mo_cnt[13:0], mo_minx, mo_maxx, mo_miny, mo_maxy,
    1'b0};    // <- bit0 保留位，必须补 0
```

RTL 注释里直接写着事故记录：”缺位时整包上移一位，OSD 解析 valid 恒 0 （永远 LOST 事故根因，2026-09-20 修）”。**把教训写在案发现场**，下一个改包的人第一眼就能看到。

12.3 控制路径：缓存 + 帧拍消费

23bit 控制包（valid+cx+cy）跨域后进 ctrl_hold 缓存，ctrl_vld_hold 标记有效；帧拍 tick_pwm 到达时消费一次。这样”包先到/帧拍先到”两种顺序都能正确工作，代价是控制滞后 1 帧——设计行为，闭环增益公式里已经算过它。

12.4 声光反馈（给嵌入式方向的接口）

`led_buzzer`：锁定亮绿灯、丢失红灯 1Hz 闪、**锁定沿**（从丢到锁）蜂鸣 200ms。演示时评委不用盯着屏幕，听声音就知道系统锁上了。

13 测试平台设计

13.1 tb_osd_track：整帧逐像素

激励：`osd_coord.txt` 给出每帧的目标坐标/包围框（金标准写的），TPG 底图 + OSD 叠加后整帧转储；期望：`osd_exp*.hex` 是 Python `render_osd()` 渲染的同帧画面。比对粒度：**像素级 24bit 相等**，4 帧共 3686400 个样本。

13.2 tb_vision_top：五条整机断言

断言	意义
直通输出与输入逐拍一致	主线零扰动（显示不受影响）
第 4 帧 OSD 与金标准一致	整链绘制的像素级正确
duty_x 在 [700,1500)	云台 X 朝目标方向动
duty_y == 1500	Y 轴死区正确（误差 20 = DZ_Y 边界内）
lost_cnt == 0	无误丢帧

迭代 3：采样过早的教训

第一版在第 2 帧末读 duty，恒 1500 判 FAIL——其实云台正常，只是” 检测 (1 帧)+ 除法 + 跨域 + 消费” 共约 2 帧滞后是设计行为。修复：采样点移到第 4 帧末。**给断言选时机，要按数据通路的真实延迟账来**，不能拍脑袋。

13.3 tb_closed_loop：相机模型驱动 TPG

TPG 方块位置由 TB 相机模型实时计算：`box_x = 640 + (40 度 - theta) x 20px/度`——云台转多少，方块就往回挪多少，**和真实相机的行为同构**。60 帧判据：误差进入 ±25px 死区并保持 10 帧。实测 `settle_frame=12`（T1 控制节拍修正后；修正前为 36），逐帧轨迹在 `sim/closed_loop_trace.txt`（答辩可直接画收敛曲线）。

14 验证结果与联调复盘

14.1 回归矩阵

测试	比对粒度	结果
tb_osd_track	整帧逐像素（4 帧 3686400 样本）	PASS
tb_vision_top	5 断言 + 整帧	PASS
tb_closed_loop	在环收敛（settle_frame）	PASS：12 帧

14.2 ”永远 LOST” 事故复盘（本步骤最大缺陷）

一次事故，三个裂缝

现象：OSD 永远显示 LOST、云台 duty 恒 1500 不动、文字区坐标被污染。

根因链：vision_top 拼 69bit - 截成 67bit（valid 位丢）；TB 侧 67bit 接 68bit 端口（MSB 悬空为 Z）；osd_track 按 67bit 解包。**三处三个宽度**，每一个单看都”能用”。

连带发现：位偏移重排后，文字分支还读旧位 [66]（现在是 preset 位）而不是 [67]（valid 位）——所以出现”显示 LOST 但坐标数字正常”的灵异现象（数字用的是 [67]）。一字符修复。

修复动作：统一宽度、位偏移重排、**冻结进 IF-VISION.md**（接口唯一权威定义），此后任何位宽变更必须走接口变更记录。

为什么逐像素比对是抓这类 bug 的唯一手段

包宽错误不会让仿真崩溃——valid 恒 0 的表现只是”画面对但没框”，肉眼完全可能当成”场景里没有目标”。只有逐像素比对才把”应有框 vs 实际无框”的每个像素差摆上台面。Step4 的三个真缺陷（包宽、旧位、字段溢出）全部由它定位。

14.3 Step4 真缺陷清单

编号	现象	根因	类型
O1	永远 LOST	包宽三处不一致（69/67/68）	接口
O2	LOST 但坐标正常	word 分支读旧位 [66]	RTL
O3	框画到 (72,18)	坐标文件%02X 写 300 溢出 3 字符	TB 脚本
O4	duty 恒 1500 误报	TB 采样点早于通路真实延迟	TB

15 上板挂载说明（给上板组）

项目	内容
挂载方式 A	显示侧：官方 hdmi_mixer 之前串入 osd_track（vpix 域）
挂载方式 B	检测侧：串在检测分支出口（本工程仿真即此形态）
vpix/DE 转换	显示侧若只有 DE 信号，按 IF-OSD.md 的规则生成 sof/eol
按键	preset_idx（阈值组）/ track_en / algo_edge / 亮度对比度
观测	ILA 挂 135bit 包 + O_status；duty_dbg 看云台

答辩演示建议

画面里的三行 OSD 本身就是”评分表逐条自证”：L1 证明扩展（1）叠加与进阶（2）坐标标注，D 字段证明进阶（3）延迟可视化，M 字段证明进阶（4）运动检测，B/C 字段证明扩展（4）实时调节——不需要额外口头解释。

16 小结：Step4 的可复用产出

- **整机顶层 vision_top**：检测/OSD/云台/声光四合一，上板即用；

- **一套 OSD 叠加层**：三行文本、双框、跑马灯、双缓冲，全部逐像素验证；
- **逐像素金标准比对方法**：此后任何绘制改动都有回归兜底；
- **接口治理范例**：135bit 包唯一权威定义 + 变更记录制度；
- **收敛证据链**：closed_loop_trace.txt 逐帧轨迹，答辩直接可画。

下一步（Step5 预告）

基础与进阶都通了，Step5 把评分表剩下的扩展项逐个补齐：多算法切换、运动检测、亮度对比度、声光反馈——每个都带位精确验证，并最终把回归扩到 16 个测试全绿。

结语

总结与下一步

Step4 学到了什么

- 点阵字库原理与生成式字库（cv2 Hershey 渲染）
- vpix 流上的 x/y 计数与叠加判决（文字/准星/双框/优先级）
- 二进制转 ASCII（常量除法）与 20 字符文本缓冲
- 影子寄存器 + 帧首换装：结构上消灭撕裂
- 135bit 目标包位图与接口单一权威定义
- 整机集成：三时钟域、直通零扰动、包与帧拍异序消费
- 逐像素金标准比对方法学

下一步学什么

- **Step5 拓展功能集**：把评分表上的进阶/扩展项逐个补齐
  - 多算法切换（边缘检测并行分支 + 帧边界无缝切换）
  - 帧差运动检测与橙色包围框
  - 亮度/对比度实时调节、声光反馈、三行 OSD 与跑马灯
- 对应文档：`03_vision/step5_expansion/`
- 对应金标准：`tools/golden_model.py::frame_diff_motion`